

HACKATHON PROBLEM STATEMENT

Codebase Intelligence Agent

AI-Powered System for Understanding Large Code Repositories

SCENARIO: You joined a startup last week. Your senior dev just quit. There's a 40,000-line production repo, an urgent bug report, and zero documentation. Where do you even start?

1. Problem Description

Modern software projects contain large and complex codebases with deeply nested folders, multiple modules, hidden dependencies, and minimal documentation.

When developers join a new project, contribute to open source, inherit an existing system, or debug legacy applications, they waste significant time trying to understand:

- Where execution starts
- How files are connected to each other
- Which modules are critical to the system
- Where business logic actually lives
- How data flows end-to-end through the system

Traditional tools like IDEs and documentation help with navigation, but do not explain architecture, execution flow, or reasoning behind file importance. As project size grows, onboarding time increases and productivity collapses.

2. Objective

Build an AI-powered Codebase Intelligence Agent that accepts a GitHub repository URL and automatically generates a structured, human-readable understanding of the project — helping developers onboard to unfamiliar codebases in minutes instead of days.

3. Mandatory Features

All teams must implement the following three core features:

Icon	Feature	Description
------	---------	-------------

M1	Folder Structure Analysis	Analyze repository folder hierarchy and explain the purpose of each major directory in plain English.
M2	Entry Point Detection	Auto-detect the project starting point (e.g., server.js, main.py, index.js) and describe the initial execution flow.
M3	Dependency Mapping	Detect file relationships using imports/exports to show how modules interact internally (e.g., route -> controller -> service -> model).

Expected Output — M1: Folder Structure

```
src/
  controllers/ → Handles incoming requests and business logic
  models/      → Manages database schemas and queries
  routes/      → Defines API endpoints and maps them to controllers
  middleware/  → Preprocesses requests (auth, logging, validation)
```

Expected Output — M2: Entry Point Detection

```
Entry Point:  server.js

Execution Flow:
  server.js loads environment variables
  → Connects to MongoDB via db.config.js
  → Registers Express middleware
  → Mounts API routes from /routes/index.js
  → Starts listening on PORT 3000
```

Expected Output — M3: Dependency Mapping

```
auth.routes.js
├── auth.controller.js
│   ├── user.service.js
│   └── user.model.js
```

4. Bonus Features

Implement any of the following to earn additional points:

- B1 — Critical File Identification: Highlight files that play major roles in project execution (auth, DB config, API controllers).
- B2 — Execution Flow Explanation: Show runtime request flow for major operations (e.g., Request → Route → Controller → Service → Database).

- B3 — Intelligent Repository Summary: Generate a high-level summary including tech stack, architecture style, and key design decisions.

5. Input & Constraints

- Input method: GitHub repository URL only.
- Supported project types: Node.js, React, Python, Java, or full-stack projects.
- AI stack: Any LLM API of your choice (Claude, GPT-4, Gemini, open-source, etc.).
- Language & framework: No restrictions — build with whatever you know best.

6. Judging Criteria

Criteria	Weight
Accuracy of analysis output	30%
Mandatory feature completeness (M1, M2, M3)	30%
Demo quality and usability	20%
Innovation / bonus features	20%

7. Expected Deliverables

- A working demo accepting a GitHub URL as input.
- Output covering all three mandatory features (M1, M2, M3).
- A short 5-minute presentation explaining your approach and demo.
- GitHub repository of your own solution.

Note: Judges will test your solution against an unseen GitHub repository during evaluation. Build for generalizability, not just your demo repo.